

VAULT — Fault-Tolerant Distributed Object Storage

Hackathon MVP source code demonstrating replication, concurrent writes, node failure and recovery, checksum-based integrity verification, corruption detection, and automatic replica repair.

How to run

Save the code as **vault.py**, then run **python vault.py** in the VS Code terminal.

Source Code

```
import os
import hashlib
import threading
import time
import random
from concurrent.futures import ThreadPoolExecutor

class ObjectData:
    def __init__(self, key, data):
        self.key = key
        self.data = data
        self.checksum = self.calculate_checksum(data)
        self.version = 1
        self.created_at = time.time()

    @staticmethod
    def calculate_checksum(data):
        return hashlib.sha256(data.encode()).hexdigest()

class StorageNode:
    def __init__(self, node_id):
        self.node_id = node_id
        self.storage = {}
        self.alive = True
        self.lock = threading.Lock()

    def write(self, obj):
        with self.lock:
            if not self.alive:
                return False
            self.storage[obj.key] = ObjectData(obj.key, obj.data)
            self.storage[obj.key].version = obj.version
            return True

    def read(self, key):
        with self.lock:
            if not self.alive:
                return None
            return self.storage.get(key)

    def corrupt(self, key):
        with self.lock:
            if key in self.storage:
                self.storage[key].data = "CORRUPTED_DATA"

    def get_keys(self):
        with self.lock:
            return list(self.storage.keys())

class Vault:
    def __init__(self, replication_factor=3):
        self.nodes = {}
        self.replication_factor = replication_factor
        self.metadata = {}
        self.metadata_lock = threading.Lock()

    def add_node(self, node_id):
        self.nodes[node_id] = StorageNode(node_id)
        print(f"[+] Node {node_id} added")

    def fail_node(self, node_id):
        if node_id in self.nodes:
            self.nodes[node_id].alive = False
            print(f"[X] Node {node_id} FAILED")

    def recover_node(self, node_id):
        if node_id in self.nodes:
            self.nodes[node_id].alive = True
            print(f"[+] Node {node_id} RECOVERED")
```

```

def choose_nodes(self, key):
    available = [n for n in self.nodes.values() if n.alive]
    if not available:
        return []
    random.seed(hash(key))
    random.shuffle(available)
    return [n.node_id for n in available[:self.replication_factor]]

def put(self, key, data):
    obj = ObjectData(key, data)
    replica_nodes = self.choose_nodes(key)

    if len(replica_nodes) < self.replication_factor:
        print(f"[!] Only {len(replica_nodes)}/{self.replication_factor} replicas available")

    successful = sum(
        self.nodes[node_id].write(obj)
        for node_id in replica_nodes
    )

    if successful:
        with self.metadata_lock:
            self.metadata[key] = replica_nodes
        print(f"[WRITE] {key} -> {replica_nodes}")
        return True

    print(f"[FAIL] Could not store {key}")
    return False

def get(self, key):
    with self.metadata_lock:
        replica_nodes = self.metadata.get(key, [])

    valid = []

    for node_id in replica_nodes:
        obj = self.nodes[node_id].read(key)
        if obj:
            checksum = ObjectData.calculate_checksum(obj.data)
            if checksum == obj.checksum:
                valid.append(obj)
            else:
                print(f"[CORRUPTION] {key} on {node_id}")

    if not valid:
        print(f"[READ] {key} -> NO VALID REPLICA")
        return None

    valid.sort(key=lambda x: x.version, reverse=True)
    print(f"[READ] {key} -> SUCCESS")
    return valid[0].data

def repair(self, key):
    with self.metadata_lock:
        replica_nodes = self.metadata.get(key, [])

    source = None

    for node_id in replica_nodes:
        obj = self.nodes[node_id].read(key)
        if obj and ObjectData.calculate_checksum(obj.data) == obj.checksum:
            source = obj
            break

    if not source:
        print(f"[REPAIR] No valid copy for {key}")
        return

    for node_id in replica_nodes:
        node = self.nodes[node_id]
        obj = node.read(key)

        valid = (
            obj is not None and
            ObjectData.calculate_checksum(obj.data) == obj.checksum
        )

        if node.alive and not valid:
            node.write(source)

    print(f"[REPAIR] {key} repaired/verified")

def verify_integrity(self):
    print("\n===== INTEGRITY CHECK =====")

    for node_id, node in self.nodes.items():
        if not node.alive:
            print(f"{node_id}: OFFLINE")
            continue

        for key in node.get_keys():
            obj = node.read(key)
            ok = (

```

```

        obj is not None and
        ObjectData.calculate_checksum(obj.data) == obj.checksum
    )
    print(f"{node_id} -> {key}: {'OK' if ok else 'CORRUPTED'}")

print("=====")

def concurrent_write_test(vault):
    def write_object(i):
        vault.put(f"file_{i}", f"Distributed object {i}")

    with ThreadPoolExecutor(max_workers=5) as executor:
        executor.map(write_object, range(1, 11))

def main():
    vault = Vault(replication_factor=3)

    for i in range(1, 6):
        vault.add_node(f"Node-{i}")

    vault.put("student.txt", "Hello from Vault!")
    vault.put("project.txt", "Fault tolerant object storage")
    vault.put("hackathon.txt", "PROMPT-A Hackathon")

    concurrent_write_test(vault)

    vault.verify_integrity()

    vault.fail_node("Node-2")
    vault.get("student.txt")

    replicas = vault.metadata.get("student.txt", [])
    if replicas:
        vault.nodes[replicas[0]].corrupt("student.txt")

    vault.verify_integrity()
    vault.get("student.txt")
    vault.repair("student.txt")

    vault.recover_node("Node-2")
    vault.verify_integrity()

    print("\nVAULT DEMO COMPLETE")

if __name__ == "__main__":
    main()

```